

ZEMEN TECH

ZemenServe

Hotel & Restaurant F&B Management System

Technical Build Specification

Prepared for automated development in Antigravity IDE

Offline, LAN-only, native Windows desktop system

A Zemen Tech Product — Built for Ethiopian hotels and restaurants

1. Project overview

Product name: ZemenServe

Publisher: Zemen Tech

Target market: Hotels and restaurants in Ethiopia

Category: Offline point-of-sale and food & beverage (F&B) control system

ZemenServe is a native Windows desktop system that lets hotel and restaurant staff take orders, track them from cashier to kitchen in real time, control food and beverage inventory at the ingredient level, and produce a daily profit report — entirely on a local network, with no internet connection required.

Build instruction for the AI agent (Antigravity IDE)

You are building ZemenServe end to end: solution structure, database, both desktop applications, real-time communication layer, and PDF reporting. Follow this document as the authoritative specification. Where a decision is not specified, choose the simplest option consistent with the constraints in Section 3 (fully offline, free/open-source tools only, native Windows desktop apps).

2. Core features

2.1 Ordering & order tracking

A waiter takes the order from the customer and relays it verbally to the cashier. The cashier enters the order into the Cashier app. The order appears immediately on the Kitchen app screen. The chef prepares the order and updates its status. No customer-facing device or app is required — the waiter never touches the software.

- Order statuses: **Pending** → **Preparing** → **Ready** → **Served**
- Order push from Cashier to Kitchen must be real-time (not polling) — target under 1 second on LAN
- Kitchen status updates push back to the Cashier app so staff know when to collect/serve

2.2 Inventory / F&B control

Every menu item has a **recipe**: a list of raw ingredients and the quantity each dish consumes (e.g. Beef Burger = 200g beef + 1 bun + 30g cheese + 50ml sauce). When an order is placed, the system automatically deducts these quantities from ingredient stock in the same database transaction as the order.

- Ingredient-level stock levels, updated automatically per order
- Low-stock alert when an ingredient falls below a configurable threshold
- Cost per dish is computed from the recipe's ingredient costs — this feeds the daily report

2.3 Digital menu (standalone, independent module)

The digital menu is a **separate, independent module** and must not be wired into the ordering flow. Its only job is to display available food items (name, price, category, image, available/unavailable) for reference or display purposes. Build it as its own screen/data table so it can be swapped out or extended later without touching the ordering system.

2.4 Daily report

At the end of the day, the Cashier app generates a **PDF report** for the manager — not a live dashboard. The report is saved locally on the Cashier PC.

- Total revenue for the day
- Total cost of goods sold (COGS), computed from recipe ingredient costs consumed
- Net profit (Revenue – COGS)
- Item-by-item sales breakdown

3. Non-negotiable constraints

- **No internet dependency.** The system must run entirely over a local wired LAN (Ethernet cable, no router required for 2 machines).
- **No manually-run server.** Neither PC's user should ever open a terminal or start a server process. Each app is a double-click executable that starts everything it needs internally on launch.
- **Two machines:** one Cashier PC, one Kitchen PC, each running its own native desktop app.
- **Fast.** All local operations and LAN order push should feel instant to the user.
- **Zero software cost.** Every tool, library, and framework used must be free for a small commercial business (see Section 6 for confirmed licensing).
- **Native Windows desktop apps** — not a website, not a browser-based app.

4. System architecture

Two independent .NET desktop applications, each packaged as a single self-contained .exe. There is no separately managed server process — the Cashier app hosts a lightweight real-time server internally, inside its own process, started automatically the moment the app opens.

Cashier app (host)

- WPF user interface: order entry, cart, checkout, digital menu screen, inventory view, report screen
- Owns the local SQLite database (single file, no database server)
- Hosts a self-hosted SignalR hub (via Kestrel) that starts automatically on app launch — this is what pushes orders to the Kitchen app in real time
- Generates the daily PDF report with QuestPDF

Kitchen app (client)

- WPF user interface: live order queue, status update buttons
- Connects automatically as a SignalR client to the Cashier app's local IP address on launch
- No local database needed — it is a thin real-time display client

Network setup: the Cashier PC is given a fixed static local IP (e.g. 192.168.1.10). The Kitchen app is configured with this IP (via a simple settings file) so it always finds the Cashier app on launch. The two PCs are connected by a single Ethernet cable, or through a basic unmanaged switch if more devices are added later.

5. Technology stack

Layer	Tool	Purpose
Language	C# / .NET 8	Core programming language and runtime
UI framework	WPF	Desktop UI for both apps; supports MVVM and touch-friendly layouts
Database	SQLite (Microsoft.Data.Sqlite + EF Core)	Local, file-based, zero-config — no database server process
Real-time sync	Self-hosted SignalR (ASP.NET Core / Kestrel, embedded)	Pushes orders from Cashier to Kitchen instantly; runs inside the Cashier .exe, starts automatically
PDF report	QuestPDF	Generates the formatted daily report
IDE	Visual Studio Community / Antigravity IDE	Development environment
Packaging	dotnet publish (self-contained, single file)	Produces one .exe per app with the .NET runtime bundled in — true double-click install

Solution structure

One Visual Studio solution, three projects:

- **ZemenServe.Shared** (class library) — data models and SignalR message contracts, referenced by both apps
- **ZemenServe.Cashier** (WPF) — order entry, database, SignalR host, PDF report generation
- **ZemenServe.Kitchen** (WPF) — order queue display, SignalR client

6. Licensing confirmation — all tools are free

Tool	License status
.NET 8 SDK	Free, Microsoft, MIT-licensed
Visual Studio Community	Free for individual developers and small teams
WPF	Included free with .NET, no separate license
SQLite	Free, public domain
Entity Framework Core	Free, open-source (MIT)
SignalR / ASP.NET Core	Free, open-source, part of .NET
QuestPDF	Free under the Community License for individuals and businesses under USD 1,000,000 annual revenue — covers this project

Total software cost: \$0.

7. Database schema (SQLite, owned by Cashier app)

Table	Key fields
menu_items	id, name, price, category, image_path, is_active
ingredients	id, name, unit, cost_per_unit, stock_quantity, low_stock_threshold
recipes	id, menu_item_id (FK), ingredient_id (FK), quantity_required
orders	id, created_at, status, total_amount, cashier_note
order_items	id, order_id (FK), menu_item_id (FK), quantity, unit_price_at_sale
inventory_log	id, ingredient_id (FK), change_amount, reason, timestamp

Relationship notes: a menu_item has many recipes (one row per ingredient it consumes). An order has many order_items. Placing an order inserts into orders and order_items, then for each order_item, looks up its recipe rows and deducts the corresponding quantity from ingredients.stock_quantity, logging each deduction to inventory_log in the same transaction.

8. Real-time communication contract

SignalR hub hosted inside the Cashier app, e.g. at `http://<cashier-ip>:5000/orderhub`. Minimum events:

Event	Direction	Purpose
NewOrder	Cashier → Kitchen	Fired when a new order is submitted; payload includes order id, items, quantities, timestamp
OrderStatusChanged	Kitchen → Cashier	Fired when the chef updates an order's status (Preparing/Ready/Served)
ConnectionRestored	Kitchen → Cashier	Fired automatically by SignalR's built-in reconnect handling after a network interruption

9. Implementation plan

#	Phase	Description
1	Solution & schema	Create the 3-project solution; define EF Core models and SQLite migrations for all tables in Section 7
2	Cashier UI	Build order entry screen: menu list, cart, submit order, saves to SQLite
3	SignalR host	Add the self-hosted SignalR hub inside Cashier app startup code, started automatically when the app opens
4	Kitchen app	Build order queue UI; connect as SignalR client to the Cashier's configured IP on launch; display incoming orders and status buttons
5	Inventory logic	Implement recipe-based stock deduction inside the order-submit transaction; add low-stock alerting
6	Digital menu module	Build as a standalone screen/data table, independent of the ordering flow

#	Phase	Description
7	PDF report	Build the QuestPDF daily report template: revenue, COGS, profit, item breakdown
8	LAN testing	Test both apps across two physical PCs over a real Ethernet connection with a static IP on the Cashier PC
9	Packaging	dotnet publish both apps as self-contained single-file executables; install one on each PC

ZemenServe — a Zemen Tech product for Ethiopian hotels and restaurants.